

Lv5 · API & MCP

AI 앱 만들기 — Claude API + Model Context Protocol 실전

총 소요: 약 75분 (강의 60분 + Q&A 15분)

구성: 12 슬라이드 · 3 원칙 + 4 시나리오 + 다음 단계

대상: Lv1~Lv4 이수자 또는 개발 경험자

출처: Anthropic Building with Claude API + Model Context Protocol 공식 문서

실무: 한국 중소기업 사내·고객·콘텐츠·MCP 4 시나리오

Slide 01 · 인트로

3분

01/12

강사 멘트

"Lv4까지는 Claude를 컴퓨터에 살게 했다면, 오늘은 **회사 앱·시스템에 살게** 합니다. 채팅창에서 API로의 전환은 단순한 기술 변화가 아니라 **비즈니스 구조 변화**입니다."

도입 질문

"여러분 회사 시스템 중 직원들이 매일 같은 정보를 찾느라 시간을 잃는 곳 1개를 떠올려 보세요."

→ 로드맵

4 블록

- Block A — API/MCP 철학 (15분)
- Block B — Claude API 기초 + 비용 절약 (25분)
- Block C — MCP 4 시나리오 (25분)
- Block D — 운영 모범 + 로드맵 (10분)

→ 원칙 1

철학적 차이

챗봇은 한 사람 한 세션. API는 무한 동시·자동화·통합. UI는 우리가 만들고, Claude는 brain만.

풍부한 사례

CASE 1 · 사이트 챗봇 비교

챗봇: 사용자가 직접 claude.ai 가서 질문

API: 우리 사이트 안에 채팅 위젯 + 우리 제품 정보 system prompt 주입 → 항상 우리 제품 답변

CASE 2 · 대량 처리

챗봇: 1만 개 제품 설명 작성 = 1만 번 사람 입력

API: 한 번 코드로 1만 개 자동 처리, 비용은 토큰 기반

CASE 3 · 비용 차이

Claude Pro \$20/월 = 한 사람 무제한

API: 토큰당 과금. Sonnet 4.6 입력 \$3/Mtok, 출력 \$15/Mtok

→ 가벼운 사용은 챗봇이, 대규모 자동화는 API가 저렴

CASE 4 · 데이터 보안

API Standard: 학습에 안 씀. Enterprise: 데이터 위치 선택. ChatGPT API와 동일 수준의 보안 정책.

CASE 5 · 한국 사례

인천 K무역회사: 처음엔 Claude Pro 5명 사용. 6개월 후 사내 모든 직원이 사용 → API + 자체 챗봇 UI로 전환. 비용 1/3 + 통합 ↑.

예상 질문

Q. "그럼 챗봇은 안 쓰나요?"

A. 사람의 일상 대화는 챗봇이 최고. API는 시스템 통합·자동화·대량 처리용. 둘 다 회사에서 쓴다.

→ MCP

왜 만들었는가

Anthropic이 2024년 11월 발표한 **오픈 표준 프로토콜**. 의도: "AI ↔ 도구 연결을 USB-C처럼 표준화". Apple Lightning 시대를 끝내자.

풍부한 사례

CASE 1 · 기존 문제

GitHub 연결 코드 1,000줄, Slack 연결 코드 다시 1,000줄, Notion 다시 1,000줄. 매번 새로 만들기. 새 AI(GPT, Gemini)가 나오면 또 새로.

CASE 2 · MCP 이후

한 번 만든 MCP 서버는 모든 AI에서 호환. Claude, GPT, Gemini, Cursor 등 어디서든 동일하게 작동.

CASE 3 · 즉시 사용 가능

```
npm install -g @modelcontextprotocol/server-github
npm install -g @modelcontextprotocol/server-postgres
npm install -g @modelcontextprotocol/server-slack
# 수십 개 공식 서버 이미 제공
```

CASE 4 · 자체 MCP 서버

회사 ERP·CRM 연결: Python 100줄 미만. Anthropic이 공식 SDK 제공. 빌드·배포 30분.

CASE 5 · 사내 시너지

한 팀이 만든 MCP를 사내 GitHub에 올리면 다른 팀이 5분 만에 사용. 회사 자산화 ↑.

실시간 데모 — 가장 중요한 5분

강사가 노트북에서 실제 코드 6줄 실행. 청중에게 보여주는 것이 핵심.

풍부한 사례 (실전 패턴)

CASE 1 · 첫 호출

```
import anthropic
client = anthropic.Anthropic() # ANTHROPIC_API_KEY 자동 사용
msg = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "안녕"}]
)
print(msg.content[0].text)
```

CASE 2 · System Prompt

```
system="너는 우리 회사 전용 비서다. 회사 매뉴얼만 참고."
messages=[{"role": "user", "content": "휴가 신청법?"}]
```

CASE 3 · 다중 턴 대화

```
history = [
    {"role": "user", "content": "매출 보고서 어떻게?"},
    {"role": "assistant", "content": "..."},
    {"role": "user", "content": "엑셀로 내보내려면?"} # 컨텍스트 유지
]
```

CASE 4 · Streaming (실시간 응답)

```
with client.messages.stream(...) as stream:
    for text in stream.text_stream: print(text, end="")
```

CASE 5 · 에러 처리

```
try:
    msg = client.messages.create(...)
except anthropic.RateLimitError:
    time.sleep(2); retry()
except anthropic.APIError as e:
    log(e); fallback_to_static_answer()
```

Q. "키 관리는?"

A. 환경변수 또는 secrets manager (AWS Secrets, GCP Secret). 코드에 절대 X.
.gitignore로 .env 제외.

→ 비용 70% 절약

Prompt Caching 핵심

긴 컨텍스트(매뉴얼·코드베이스·문서)는 매 호출마다 재전송하면 비용 폭발. **cache_control** 추가로 5분 캐시 → 다음 호출 1/10 비용.

풍부한 사례

CASE 1 · 캐시 전후 비교

매뉴얼 50,000 토큰 사용자 100명 동시 질문:

- 캐시 없음: $50,000 \times 100 = 5,000,000$ 토큰 = \$15
- 캐시 적용: 50,000 (작성) + $5,000 \times 99$ (히트) = \$4.5 → **70% ↓**

CASE 2 · 캐시 위치

```
system=[{
  "type": "text",
  "text": "우리 회사 매뉴얼 100페이지...",
  "cache_control": {"type": "ephemeral"} # ← 여기
}]
```

CASE 3 · Batch API + 캐시

비실시간 대량 작업: Batch API (50% ↓) + Prompt Caching (70% ↓) 합치면 비용 약 85% ↓.

CASE 4 · 모델 선택 전략

간단한 분류·요약은 Haiku 4.5 (Sonnet 1/3 비용). 복잡한 추론만 Opus. 라우터 패턴으로 자동 분기.

CASE 5 · 비용 모니터링

Anthropic Console 무료 대시보드 → 모델별·일별·request_id별 비용 확인. 한도 초과 시 알림.

핵심 — "데이터를 AI에게 가져오기" → "AI를 데이터로 보내기"

사람이 ERP 데이터를 export해서 AI에 붙여넣는 시대 끝. MCP 서버로 Claude가 직접 ERP에 접근.

풍부한 사례

CASE 1 · ERP 매출

경리부장이 매일 "이번 주 매출" 물음 → 공식 MCP postgres-server + 회사 DB → Claude가 SQL 실행 → 표.

CASE 2 · 그룹웨어 결재

```
// MCP 서버 함수  
get_pending_approvals(user_id) → JSON[]  
get_document(doc_id) → text
```

CASE 3 · CRM 응대 분석

"홍길동 고객 지난 6개월 응대 패턴" → MCP가 CRM API 호출 → 답변 + 다음 액션 제안.

CASE 4 · 사내 위키 RAG

벡터 DB(Chroma/Qdrant) + Anthropic embedding → 의미 검색 → 출처 첨부. 답변 신뢰도 ↑.

CASE 5 · 권한 모델

MCP 함수 안에서 사용자 권한 체크 — Claude가 그 사람 권한으로만 쿼리. PII 마스킹 필수.

→ 고객 응대

외부 노출 챗봇 — 신중하게

사내 도구는 일부 환각·실수 허용. 고객 응대는 다르다. **system prompt + RAG + 가드레일** 3중 보호.

풍부한 사례

CASE 1 · 사이트 챗봇 system prompt

"너는 ABC 인테리어 안내 비서다. 우리 제품·서비스만 답변.

- 다른 회사 추천 X
- 견적·계약은 사람 상담 안내
- 답변에 항상 출처 명시"

CASE 2 · 이메일 1차 응대

메일 받으면 → API 분류 → 단순 질문은 자동 답장, 복잡한 건 사람 알림. 응답 시간 1시간 → 1분.

CASE 3 · 음성 상담

전화 → Whisper(STT) → Claude → ElevenLabs(TTS). 전화 첫 30초 자동화. 사람 응대는 복잡한 케이스만.

CASE 4 · 앱 내 도움말

"?"버튼 클릭 → 현재 화면 컨텍스트 자동 추가 → Claude 답변. UI 도움말 비용 ↓.

CASE 5 · 안전장치

출력 검증 함수: 가격·약속 제시 X, 부적절 단어 차단. 사고 시 자동 사람 전환.

실시간 vs Batch

실시간이 필요 없으면 무조건 Batch API. 50% 절약 + 한 번에 10만 건 처리.

풍부한 사례

CASE 1 · 상품 설명 1만개

제품 스펙 CSV → Batch API → 24시간 안에 1만 개 카피 + SEO 메타. 비용 \$20 → \$10.

CASE 2 · 다국어 번역

한국어 → 영·일·중·베트남. 톤별(비즈니스·친근) 옵션. 100건 동시.

CASE 3 · Vision + 이미지 캡션

제품 이미지 100장 → Claude Vision → SEO alt text + 캡션. 검색 노출 ↑.

CASE 4 · Batch API 비용 계산

실시간: \$3/Mtok input
Batch: \$1.5/Mtok input
+ Caching 적용 시: \$0.45/Mtok

CASE 5 · 품질 관리

샘플 100건 → 사람 검토 → 패턴 보강 → 전체 실행. 사람이 처음 100건만 본다.

Python 100줄로 시작

Anthropic 공식 Python SDK가 거의 모든 보일러플레이트 처리. 한 함수 = 한 도구.

풍부한 사례

CASE 1 · 매출 조회 MCP 구조

```
from mcp.server import Server
server = Server("sales-mcp")

@server.tool()
def get_sales(month: str, region: str) -> dict:
    """월·지역별 매출 조회"""
    rows = db.query(f"SELECT ... WHERE ...")
    return {"total": ..., "by_product": [...]}
```

CASE 2 · 권한·보안

```
@server.tool()
def find_customer(query: str, user_id: str):
    if not has_permission(user_id, "customer_read"):
        return {"error": "권한 없음"}
    # PII 마스킹 후 반환
    ...
```

CASE 3 · 쓰기 작업

```
@server.tool()
def create_invoice(data: dict, confirm: bool = False):
    if not confirm:
        return {"preview": ..., "needs_confirm": True}
    # 사람 확인 후 실행
    ...
```

CASE 4 · 사내 GitHub 공유

MCP 서버 = 일반 Python 패키지. 사내 PyPI에 올리면 다른 팀 5분 설치.

CASE 5 · 표준화

한 번 만든 MCP는 Claude Desktop, API 앱, Cursor, Windsurf 등 어디서든 동일 작동. 회사 자산.

→ 운영 모범

PoC와 운영은 다르다

예쁘게 동작하는 데모는 쉬움. 매일 1000명 사용은 운영 체크 4가지 필수.

풍부한 사례

CASE 1 · Rate Limit

Anthropic SDK 자동 지수 백오프. 직접 구현 시 1s, 2s, 4s, 8s ... max 60s 권장.

CASE 2 · Prompt Caching 운영

system prompt 자주 변경 X. 변경 시 캐시 무효화 비용 고려.

CASE 3 · Observability

모든 호출 request_id 로깅 → 비용·지연·에러 추적 → 일주일 후 패턴 분석.

CASE 4 · Fallback

API 다운 시 자동 처리: ① 5초 대기 후 재시도, ② 다른 모델 (Haiku), ③ 정적 답변.

CASE 5 · 회사 가드레일

system prompt에 "금지 단어 리스트" + 출력 후 검증 함수 → 부적절 응답 자동 차단.

→ 다음 단계

오늘의 약속

"일주일 안에 회사 도구 1개 + Claude API로 작동하는 PoC 만들기." 한 번 만들면 다음은 자연스럽게.

로드맵

Lv6 · Agents & Skills: 여러 단계 자율 처리. Anthropic Building Effective Agents 백서 기반.

Lv7 · 클라우드 AI: AWS Bedrock/Vertex/Azure 운영 모범.

Q&A

Q. "비용이 무서워요"

A. 한도(spend limit) + 캐싱 + Haiku 모델 조합으로 통제. 일일 한도 \$10 같은 설정 가능.

Q. "Claude vs GPT 비교는?"

A. 코드·긴 컨텍스트·정확도는 Claude 우세. 멀티모달 일부 분야는 GPT-4o. 둘 다 쓰는 게 일반.

Q. "API + 사내 데이터 보안 우려"

A. Standard tier도 학습에 안 씬. 더 강력하면 Enterprise (데이터 위치 선택) 또는 Bedrock(Lv7).

Q. "MCP는 누가 만드나요?"

A. 회사 IT팀 또는 외주. Claude Code(Lv4)에 "MCP 서버 만들어줘"라고 시키면 1시간.